CONFORMITÉ EXPORT · DOCUMENTATION DU CHIFFREMENT

FamilyNexus – Description technique des fonctions cryptographiques

État : 31/10/2026, correspond à la version 1.2.1. Projet de document pour la déclaration relative à un moyen de cryptologie (ANSSI, https://demarche.numerique.gouv.fr/commencer/declaration-relative-a-un-moyen-de-cryptologie) et pour App Store Connect. Les points marqués [à vérifier] n'ont pas pu être confirmés à partir des sources du projet. Cette traduction est un projet : à faire relire par un locuteur natif ou un conseil juridique avant tout dépôt. Version de référence : `encryption-description-en.md` *.*

1. Produit

Nom du produit	FamilyNexus
Version	1.2.1
Identifiant	`de.inneso.familynexus`
Plateformes	iOS 15+, macOS 12+ (App Store). Autres versions (Android 8.0+, Linux, Windows, navigateur) non diffusées via l'App Store, même conception cryptographique
Fournisseur / éditeur	**Alexander Ziemer, Nahestr. 31, 55593 Rüdesheim, Germany**
Code source	Public, licence AGPL-3.0-or-later : https://github.com/carpeasrael/FamilyNexus
Fonction principale	Agenda familial, messagerie familiale et liste de courses partagée
Diffusion	Apple App Store (grand public, gratuit, sans version spécifique)

La fonction principale n'est pas la sécurité de l'information. La cryptographie protège la confidentialité et l'intégrité des messages, des entrées d'agenda partagées et des données locales de l'utilisateur. L'application n'est ni un produit d'infrastructure réseau, ni un VPN, ni une boîte à outils cryptographique, et n'offre aucune interface cryptographique accessible à l'utilisateur (les algorithmes ne peuvent être ni modifiés, ni remplacés, ni étendus).

2. Utilisation de la cryptographie

L'application utilise le protocole Matrix (https://spec.matrix.org) pour la messagerie chiffrée de bout en bout. Toutes les primitives sont des algorithmes standard ; aucun algorithme propriétaire n'est utilisé.

#	Fonction	Algorithmes	Longueur de clé	Mise en œuvre
1	Sécurité du transport vers le serveur Matrix choisi (HTTPS uniquement)	TLS 1.2 / 1.3 (échange ECDHE, AES-GCM / ChaCha20-Poly1305)	jusqu'à 256 bits	Pile TLS de l'exécution Dart (BoringSSL) [à vérifier] ; validation des certificats par le magasin de confiance du système

#	Fonction	Algorithmes	Longueur de clé	Mise en œuvre
2	Chiffrement de bout en bout : établissement de session et messages d'appareil à appareil (Olm)	Curve25519 (ECDH), Ed25519, HKDF-SHA-256, AES-256-CBC, HMAC-SHA-256	256 bits	`vodozemac` (bibliothèque Rust de la Matrix.org Foundation)
3	Chiffrement de bout en bout des messages de salon, entrées d'agenda et de liste de courses (Megolm)	AES-256-CBC, HMAC-SHA-256, Ed25519	256 bits	`vodozemac`
4	Vérification des appareils et signature croisée	Ed25519 ; vérification SAS : Curve25519 (ECDH), HKDF-SHA-256, HMAC-SHA-256, SHA-256	256 bits	`vodozemac` , Matrix Dart SDK 14.2.0
5	Sauvegarde chiffrée des clés et stockage de secrets (clé de récupération)	Curve25519 + HKDF-SHA-256 + AES-256-CBC + HMAC-SHA-256 ; AES-256-CTR + HMAC-SHA-256 ; clé de récupération aléatoire de 256 bits	256 bits	`vodozemac` , Matrix Dart SDK
6	Pièces jointes chiffrées	AES-256-CTR (clé aléatoire de 256 bits, IV de 128 bits), empreinte SHA-256	256 bits	Matrix Dart SDK / `vodozemac`
7	Base de données locale chiffrée	SQLCipher 4.x : AES-256-CBC, HMAC-SHA-512, PBKDF2-HMAC-SHA-512	256 bits	`sqflite_sqlcipher` 3.4.1 avec SQLCipher 4.10.0 (iOS et macOS) **[fournisseur cryptographique à vérifier]**
8	Clé de la base	32 octets aléatoires (générateur cryptographiquement sûr), conservés dans le trousseau du système (Keychain ; lisibles après le premier déverrouillage, uniquement sur cet appareil, jamais synchronisés ni inclus dans les sauvegardes)	256 bits	`flutter_secure_storage` 11.2.0 (chiffrement intégré au système d'exploitation)
9	Verrouillage de l'application	Authentification biométrique / code de l'appareil par le système (LocalAuthentication) ; l'application ne conserve ni code ni mot de passe propre	–	`local_auth` 3.0.2 (système d'exploitation Apple)
10	Version navigateur uniquement (hors applications App Store)	Coffre de clés dans `localStorage` : AES-256-GCM, clé dérivée du mot de passe par PBKDF2-HMAC-SHA-256 (600 000 itérations) ; base : SQLite3MultipleCiphers (WebAssembly)	256 bits	WebCrypto (`SubtleCrypto`) du navigateur

Non utilisés : séquestre ou recouvrement de clés par un tiers, protocoles d'échange de clés propriétaires, chiffrement pour la gestion des droits numériques, chiffrement de code exécutable, stéganographie, cryptographie anti-falsification.

3. Normes de référence

AES (FIPS 197), modes CBC/CTR (NIST SP 800-38A), HMAC (RFC 2104), SHA-256/512 (FIPS 180-4), HKDF (RFC 5869), PBKDF2 (RFC 8018), X25519 (RFC 7748), Ed25519 (RFC 8032), TLS 1.2/1.3 (RFC 5246 / RFC 8446). Les protocoles Olm et Megolm sont spécifiés publiquement par la Matrix.org Foundation et combinent ces primitives standard.

4. Flux de données

1. L'utilisateur se connecte à un serveur Matrix de son choix en HTTPS.
2. Messages, entrées d'agenda partagées et listes de courses sont chiffrés sur l'appareil avant l'envoi ; le serveur ne stocke que du texte chiffré.
3. Les entrées d'agenda privées ne quittent les appareils de l'utilisateur que dans un salon chiffré privé de son propre compte.
4. Les données locales sont stockées dans une base SQLCipher ; la clé reste dans le trousseau de l'appareil.
5. L'éditeur n'exploite aucun serveur, ne reçoit aucune donnée et n'utilise aucune télémétrie.